

Frontend Pages CRUD Analysis

This document analyzes the CRUD (Create, Read, Update, Delete) operations available in each frontend page.

CRUD Legend

- ☒ **Implemented** - Functionality exists
- **✗ Missing** - Functionality not implemented
- ☐ **Partial** - Some functionality exists but incomplete
- N/A **Not Applicable** - Operation not needed for this resource

CRUD Operations Summary

Page	Create	Read	Update	Delete	Status
Users.tsx	✗	☒	☒	✗	☐ Partial
Products.tsx	✗	☒	☒	☒	☐ Partial
Orders.tsx	✗	☒	☒	N/A	☐ Partial
Inventory.tsx	✗	☒	☒	✗	☐ Partial
Production.tsx	✗	☒	☒	✗	☐ Partial
QualityControlDashboard.tsx	✗	☒	☒	✗	☐ Partial
SystemLogs.tsx	N/A	☒	N/A	N/A	☒ Complete
ApiKeys.tsx	☒	☒	☒	☒	☒ Complete
NotificationConfigs.tsx	☒	☒	☒	☒	☒ Complete
DataManagementDashboard.tsx	☒	☒	☒	☒	☒ Complete
CustomerCRM.tsx	✗	☒	☒	✗	☐ Partial
EmployeeDashboard.tsx	✗	☒	☒	✗	☐ Partial

Detailed CRUD Analysis

1. Users.tsx (/api/users)

Current Implementation:

```
// ☒ READ - fetchUsers()
const fetchUsers = async () => {
  const response = await apiService.getUsers();
```

```

    setUsers(response.data);
  };

  // ☒ UPDATE - handleAction()
  const handleAction = async (userId: string, action: string, reason?: string) => {
    // approve, reject, suspend, activate, deactivate
  };

  // ☒ CREATE - Missing
  // No "Add User" button or dialog

  // ☒ DELETE - Missing
  // No delete button or functionality

```

Missing Operations:

- **Create:** No form to create new users
- **Delete:** No delete functionality

Recommendation:

Add "Add User" button with a dialog form and "Delete" button for each user row.

2. Products.tsx (/api/products)

Current Implementation:

```

// ☒ READ - loadProducts()
const loadProducts = async () => {
  const response = await apiService.getProducts();
  setProducts(response.data);
};

// ☒ UPDATE - handleProductUpdate()
const handleProductUpdate = async (productId: string, updates: any) => {
  await apiService.updateProduct(productId, updates);
  await loadProducts();
};

// ☒ DELETE - handleProductDelete()
const handleProductDelete = async (productId: string) => {
  await apiService.deleteProduct(productId);
  await loadProducts();
};

// ☒ CREATE - Missing
// No "Add Product" button or dialog

```

Missing Operations:

- **Create:** No form to add new products

Recommendation:

Add "Add Product" button with a dialog form similar to the Edit dialog.

3. Orders.tsx (/api/orders)

Current Implementation:

```
// ☒ READ - loadOrders()
const loadOrders = async () => {
  const response = await apiService.getOrders();
  setOrders(response.data);
};

// ☒ UPDATE - handleStatusUpdate()
const handleStatusUpdate = async () => {
  await apiService.updateOrderStatus(selectedOrder.id, newStatus);
  await loadOrders();
};

// ☒ CREATE - Missing
// No "Create Order" button or dialog

// N/A DELETE - Not applicable
// Orders typically cannot be deleted, only cancelled
```

Missing Operations:

- **Create:** No form to create new orders (orders are typically created by customers)

Note:

Orders are usually created through the customer-facing shop, not by admins. However, a "Create Order" feature could be useful for phone/email orders.

4. Inventory.tsx (/api/inventory)

Current Implementation:

```
// ☒ READ - loadInventory()
const loadInventory = async () => {
  const response = await apiService.getInventory();
  setInventory(response.data);
};

// ☒ UPDATE - handleInventoryUpdate()
const handleInventoryUpdate = async (itemId: string, updates: any) => {
  await apiService.updateInventoryItem(itemId, updates);
  await loadInventory();
};

// ☒ CREATE - Missing
// No "Add Inventory Item" button or dialog

// ☒ DELETE - Missing
// No delete functionality
```

Missing Operations:

- **Create:** No form to add new inventory items
- **Delete:** No delete functionality

5. Production.tsx (/api/production)

Current Implementation:

```
// ☒ READ - loadProduction()
const loadProduction = async () => {
  const response = await apiService.getProductionCycles();
  setProductionCycles(response.data);
};

// ☒ UPDATE - handleProductionUpdate()
const handleProductionUpdate = async (cycleId: string, updates: any) => {
  await apiService.updateProductionCycle(cycleId, updates);
  await loadProduction();
};

// ☒ CREATE - Missing
// No "Start New Cycle" button or dialog

// ☒ DELETE - Missing
// No delete functionality
```

Missing Operations:

- **Create:** No form to start new production cycles
 - **Delete:** No delete functionality
-

6. QualityControlDashboard.tsx (/api/quality)

Current Implementation:

```
// ☒ READ - loadQualityData()
const loadQualityData = async () => {
  const response = await apiService.getQualityInspections();
  setInspections(response.data);
};

// ☒ UPDATE - handleInspectionUpdate()
const handleInspectionUpdate = async (inspectionId: string, updates: any) => {
  await apiService.updateQualityInspection(inspectionId, updates);
  await loadQualityData();
};

// ☒ CREATE - Missing
// No "New Inspection" button or dialog

// ☒ DELETE - Missing
// No delete functionality
```

Missing Operations:

- **Create:** No form to create new quality inspections
 - **Delete:** No delete functionality
-

7. ApiKeys.tsx (/api/api-keys) ☒ Complete

Current Implementation:

```
// ☒ CREATE - handleCreateApiKey()
const handleCreateApiKey = async () => {
  await apiService.createApiKey(apiKeyData);
  await loadApiKeys();
};

// ☒ READ - loadApiKeys()
const loadApiKeys = async () => {
  const response = await apiService.getApiKeys();
  setApiKeys(response.data);
};
```

```
// ☒ UPDATE - handleUpdateApiKey()
const handleUpdateApiKey = async (keyId: string, updates: any) => {
  await apiService.updateApiKey(keyId, updates);
  await loadApiKeys();
};

// ☒ DELETE - handleDeleteApiKey()
const handleDeleteApiKey = async (keyId: string) => {
  await apiService.deleteApiKey(keyId);
  await loadApiKeys();
};
```

Status: ☒ Full CRUD implemented

8. NotificationConfigs.tsx (/api/notification-configs) ☒ Complete

Current Implementation:

```
// ☒ CREATE - handleCreateConfig()
const handleCreateConfig = async () => {
  await apiService.createNotificationConfig(configData);
  await loadConfigs();
};

// ☒ READ - loadConfigs()
const loadConfigs = async () => {
  const response = await apiService.getNotificationConfigs();
  setConfigs(response.data);
};

// ☒ UPDATE - handleUpdateConfig()
const handleUpdateConfig = async (configId: string, updates: any) => {
  await apiService.updateNotificationConfig(configId, updates);
  await loadConfigs();
};

// ☒ DELETE - handleDeleteConfig()
const handleDeleteConfig = async (configId: string) => {
  await apiService.deleteNotificationConfig(configId);
  await loadConfigs();
};
```

Status: ☒ Full CRUD implemented

Recommendations for Missing CRUD Operations

High Priority (Admin Pages)

1. Users.tsx

Missing: Create, Delete

Implementation:

```
// Add to component state
const [createDialogOpen, setCreateDialogOpen] = useState(false);
const [newUserData, setNewUserData] = useState({
  email: '',
  userType: 'consumer',
  // ... other fields
});

// Add create function
const handleCreateUser = async () => {
  try {
    await apiService.createUser(newUserData);
    await fetchUsers();
    setCreateDialogOpen(false);
    setSnackbar({ open: true, message: 'User created successfully', severity:
'success' });
  } catch (error) {
    setSnackbar({ open: true, message: 'Failed to create user', severity:
'error' });
  }
};

// Add delete function
const handleDeleteUser = async (userId: string) => {
  if (window.confirm('Are you sure you want to delete this user?')) {
    try {
      await apiService.deleteUser(userId);
      await fetchUsers();
      setSnackbar({ open: true, message: 'User deleted successfully', severity:
'success' });
    } catch (error) {
      setSnackbar({ open: true, message: 'Failed to delete user', severity:
'error' });
    }
  }
};
```

2. Products.tsx

Missing: Create

Implementation:

```

// Add to component state
const [createDialogOpen, setCreateDialogOpen] = useState(false);
const [newProductData, setNewProductData] = useState({
  name: '',
  sku: '',
  category: 'Whole Birds',
  // ... other fields
});

// Add create function
const handleCreateProduct = async () => {
  try {
    await apiService.createProduct(newProductData);
    await loadProducts();
    setCreateDialogOpen(false);
    setError(null);
  } catch (error) {
    setError('Failed to create product');
  }
};

```

3. Inventory.tsx

Missing: Create, Delete

Implementation:

```

// Add create function
const handleCreateInventoryItem = async (itemData: any) => {
  try {
    await apiService.createInventoryItem(itemData);
    await loadInventory();
    setSnackbar({ open: true, message: 'Inventory item created', severity:
'success' });
  } catch (error) {
    setSnackbar({ open: true, message: 'Failed to create item', severity:
'error' });
  }
};

// Add delete function
const handleDeleteInventoryItem = async (itemId: string) => {
  if (window.confirm('Delete this inventory item?')) {
    try {
      await apiService.deleteInventoryItem(itemId);
      await loadInventory();
      setSnackbar({ open: true, message: 'Item deleted', severity: 'success'
});
    } catch (error) {

```

```

        setSnackbar({ open: true, message: 'Failed to delete item', severity:
'error' });
    }
}
};

```

4. Production.tsx

Missing: Create, Delete

Implementation:

```

// Add create function
const handleCreateProductionCycle = async (cycleData: any) => {
    try {
        await apiService.createProductionCycle(cycleData);
        await loadProduction();
        setSnackbar({ open: true, message: 'Production cycle started', severity:
'success' });
    } catch (error) {
        setSnackbar({ open: true, message: 'Failed to start cycle', severity:
'error' });
    }
};

```

5. QualityControlDashboard.tsx

Missing: Create, Delete

Implementation:

```

// Add create function
const handleCreateInspection = async (inspectionData: any) => {
    try {
        await apiService.createQualityInspection(inspectionData);
        await loadQualityData();
        setSnackbar({ open: true, message: 'Inspection created', severity:
'success' });
    } catch (error) {
        setSnackbar({ open: true, message: 'Failed to create inspection', severity:
'error' });
    }
};

```

Backend API Methods Needed

The frontend `api.ts` service needs these methods for full CRUD:

```
// Users
createUser(userData: any): Promise<any>;
deleteUser(userId: string): Promise<any>;

// Products
createProduct(productData: any): Promise<any>;

// Inventory
createInventoryItem(itemData: any): Promise<any>;
deleteInventoryItem(itemId: string): Promise<any>;

// Production
createProductionCycle(cycleData: any): Promise<any>;
deleteProductionCycle(cycleId: string): Promise<any>;

// Quality
createQualityInspection(inspectionData: any): Promise<any>;
deleteQualityInspection(inspectionId: string): Promise<any>;

// Orders (optional)
createOrder(orderData: any): Promise<any>;
```

Implementation Checklist

High Priority (Admin Management Pages)

- ☐ Add "Create User" functionality to Users.tsx
- ☐ Add "Delete User" functionality to Users.tsx
- ☐ Add "Create Product" functionality to Products.tsx
- ☐ Add "Create Inventory Item" functionality to Inventory.tsx
- ☐ Add "Delete Inventory Item" functionality to Inventory.tsx
- ☐ Add "Create Production Cycle" functionality to Production.tsx
- ☐ Add "Create Quality Inspection" functionality to QualityControlDashboard.tsx

Medium Priority

- ☐ Add "Create Order" functionality to Orders.tsx (for admin phone/email orders)
- ☐ Add "Delete Production Cycle" functionality to Production.tsx
- ☐ Add "Delete Quality Inspection" functionality to QualityControlDashboard.tsx

Low Priority

- ☐ Add bulk operations (bulk delete, bulk update)
- ☐ Add import/export functionality for each resource
- ☐ Add advanced filtering and sorting

UI Components Needed

Reusable Components

1. **CreateDialog** - Generic dialog for creating new items
2. **DeleteConfirmDialog** - Confirmation dialog for delete operations
3. **FormFields** - Reusable form components (TextField, Select, etc.)

Example Create Dialog Structure

```
interface CreateDialogProps {  
  open: boolean;  
  onClose: () => void;  
  onSubmit: (data: any) => void;  
  title: string;  
  fields: FormField[];  
}
```

Summary

Current CRUD Status:

- **Full CRUD**: 3 pages (ApiKeys, NotificationConfigs, DataManagement)
- **Partial CRUD (R+U)**: 8 pages (Users, Products, Orders, Inventory, Production, Quality, CRM, Employees)
- **Read-Only**: 1 page (SystemLogs)

Missing Create Operations: 8 pages

Missing Delete Operations: 6 pages

Recommendation:

Prioritize adding **Create** functionality to admin management pages (Users, Products, Inventory, Production, Quality) as these are essential for day-to-day operations.

Notes

1. **Orders**: Typically created by customers through the shop, not by admins
2. **SystemLogs**: Read-only is appropriate (logs shouldn't be modified)
3. **Analytics Pages**: Read-only is appropriate (data is computed, not stored)
4. **Dashboard Pages**: Read-only is appropriate (displays aggregated data)